Agile Web Development with Rails 8

Part 1 - Getting Started

Chapter 1 - Installing Rails

Chapter 2 - Instant Gratification

- Use `rails new demo` to start a new app named `demo`.
- `bin/rails about` from your project directory shows a lot of information about your project. You could also `bundle exec rails about`.
- `bin/dev` will start a dev server.
- `bin/rails generate controller Say hello goodbye` will generate a `Say` controller with `hello` and `goodbye` actions.
- By default, Rails looks for templates in a file with the same name as the action it's handling. The views for say live in `app/views/say` and the `hello` action shows `hello.html.erb`. Controllers go in `app/controllers`.
- Content between `<%=` and `%>` in an erb file is interpreted as Ruby code and executed. The results are converted to a string and displayed.
- `<%` and `%>` embed code without inserting the results in the output.
- An instance variable in your controller can be declared in the `hello` method to access from the `hello.html.erb` template like `@time = Time.now`. The time is data and should be supplied to the view by the controller even if we could have embedded a `Time.now` call in the view.
- The default is for `http://localhost:3000/say/hello` to invoke the `hello` method of the `say` controller.
- You can link to `/say/goodbye` with `<%= link_to "Goodbye", say_goodbye_path %>`.

Chapter 3 - The Architecture of Rails Applications

- The *model* is responsible for maintaining the state of the application. It is more than just the data; it enforces all the business rules that apply to the data.
- The *view* is responsible for generating a user interface, normally based on data in the model. The view might present users with various ways to input data, but the view itself never handles incoming data.

- *Controllers* orchestrate the application. They receive events from the outside world (normally, user input), interact with the model, and display an appropriate view to the user.
- In a Rails application, the request is first sent to a router which identifies a particular method (called an *action*) on a controller. The action may look at data in the request, interact with the model, and/or cause other actions to be invoked. Eventually the action prepares information for the view, which renders something to the user.
- Object-relational mapping (ORM) libraries map database tables to classes. If our database has a table called **orders**, our program will have an **Order** class. Rows in the table correspond to objects of the class. Within that object, attributes are used to get and set the individual columns. Our **Order** object has methods to get and set the amount, the sales tax, etc. Our class objects will have methods that perform table-level operations also, like **find** to find by ID. Instance methods perform operations on individual rows. Active Record is the ORM in Rails.

```
order = Order.find(1)
puts "Customer #{order.customer_id}, amount=$#{order.amount}"

Order.where(name: 'dave').each do |order|
  puts order.amount
end

Order.where(name: 'dave').each do |order|
  order.pay_type = "Purchase order"
  order.save
end
```

- Active Storage allows you to attach files from cloud storage services like S3 to your Active Records.
- Action Pack provides support for both views and controllers.
- Embedded Ruby (ERB) files support embedding Ruby snippets in the view, but be judicious about putting code directly in the view layer instead of using logic in the model or controller.

Chapter 4 - Introduction to Ruby

- Everything you manipulate in Ruby is an object, and the results of those manipulations are themselves objects.

- You create objects with a *constructor*. The standard constructor is called **new**. To construct a **LineItem** class:

```
line_item_one = LineItem.new
line_item_one.quantity = 1
line_item_one.sku      = "AUTO_B_00"
line_item_one.quantity() # => 1
"dave".length            # => 4
```

- You invoke methods on objects by sending a message to the object containing the method's name along with any parameters the method may need. When an object receives a message, it looks at its own class for the method.

- Parentheses are generally optional in method calls.

- Local variables, method parameters, and method names should all start with a lowercase letter or an underscore. Instance variables begin with `@`. You separate words in a multiword method or variable name with underscores (`_`). Class names, module names, and constants must start with an uppercase letter. By convention, they use capitalization rather than underscores to distinguish the start of words within their name. Rails uses *symbols* to identify things. A symbol looks like a variable name but is prefixed with a colon (`:`). You can think of them as string literals magically made into constants.

- You define a method with `def` and end the definition with `end`. You don't need a semicolon at the end of a statement as long as each statement is on a separate line. Ruby comments start with `#` and go to the end of the line. Indentation isn't significant, and 2 spaces is standard. Ruby doesn't use braces to delimit the bodies of compound statements and definitions; you just end the body with the `end` keyword. The `return` keyword is optional and if not present the results of the last expression evaluated are returned.

- String literals can be single quoted (`'`) or double quoted (`"`). Single quoted strings involve very little processing. Double quoted strings do *substitutions* such as replacing "\n" with the newline character. Ruby also performs *expression interpolation* in double-quote strings where the sequence `#{expression}` is replaced.

- Ruby's arrays and hashes are indexed collections. They store collections of objects accessible with a key. The key is an integer for arrays and any object for hashes. Both grow as needed and can hold objects of differing types. Array literals are a set of objects between square brackets `[]`. You access an element by giving the array name followed by square brackets with an index in them.

```ruby
a = [ 1, "cat", 3.14 ]    # array with three elements
a[0]                      # access the first element => 1
a[2] = nil                # set the third element
a                         # => [ 1, "cat", nil ]
```

- *nil* is an object like any other but represents nothing.
- The `<<` method appends an item to an array.
- Ruby has a shortcut for an array of words: `a = %w[ ant bee cat dog elk ]` is equivalent to `a = ['ant', 'bee', ... ]`.
- A hash literal uses braces rather than square brackets and the literal must supply two objects for every entry: one key and one value. Keys must be unique and if you re-use a key the last assignment wins. In Rails, hashes normally use symbols as keys although they could be any object. Many hashes have been subtly modified so you can use a string or symbol interchangeably.

```ruby
inst_section = {
    :cello     => "string",
    :clarinet  => "woodwind",
    :drum      => "percussion",
    :oboe      => "woodwind",
    :trumpet   => "brass",
    :violin    => "string"
}

# Equivalently using shorthand syntax usable with symbol keys
inst_section = {
    cello:      "string",
    clarinet:   "woodwind",
    drum:       "percussion",
    oboe:       "woodwind",
    trumpet:    "brass",
    violin:     "string"
}

inst_section[:oboe]    # => "woodwind"
inst_section[:bassoon] # => nil
```

- You can omit the braces when passing hashes as the last parameter of a method call. `redirect_to action: "show", id: product.id` is actually passing a two element hash to

`redirect_to`. This is the same syntax as keyword arguments.

- The regular expression is a first class type you can express as either `/pattern/` or `%r{pattern}`. Programs typically use the match operator `=~` to test strings against regular expressions.

```
if line =~ /P(erl|ython)/
  puts "There seems to be another scripting language here"
end
```

- Ruby has **if** and **while** statements:

```
if count > 10
  puts "Try again"
elsif tries == 3
  puts "You lose"
else
  puts "Enter a number"
end

while weight < 100 and num_pallets <= 30
  pallet = next_pallet()
  weight += pallet.weight
  num_pallets += 1
end
```

- It also has **unless**, which is like **if** except it checks for the condition to be not true, and **until** which is like **while** except the loop continues until the condition evaluates to true.
- The Ruby control structures can also be used as *statement modifiers* where you place an **if**, **unless**, **while**, or **until** at the end of an expression to affect only that expression.
- Code blocks are statements between `{}` or between `do end`. Commonly braces are for single line blocks and `do end` are for multiline. To pass a block to a method, place the block after the parameters (if any) to the method. A method can invoke an associated block one or more times using **yield**. You can pass values to the block by giving parameters to **yield**. Within the block, you list the names of the arguments to receive these parameters between vertical bars `|`.

```ruby
animals = %w[ ant bee cat dog elk ]   # create an array
animals.each {|animal| puts animal }  # iterate over the contents
3.times { print "Ho! " }              # => Ho! Ho! Ho!
# The & prefix lets a method capture a block as a named parameter
def wrap &b
  print "Santa says: "
  3.times(&b)
  print "\n"
end
wrap { print "Ho! "}
```

- Control is sequential within a block or method unless an exception occurs.
- Exceptions are objects of the **Exception** class or its subclasses. The **raise** method causes an exception to be raised. Both methods and blocks of code wrapped between **begin** and **end** keywords intercept certain classes of exception using **rescue** clauses:

```ruby
begin
  content = load_blog_data(file_name)
rescue BlogDataNotFound
  STDERR.puts "File #{file_name} not found"
rescue BlogDataFormatError
  STDERR.puts "Invalid blog data in #{file_name}"
rescue Exception => exc
  STDERR.puts "General error loading #{file_name}: #{exc.message}"
end
```

- **rescue** clauses can be directly placed on the outermost level of a method definition without needing to enclose the contents in a **begin/end** block.
- Ruby has two basic concepts for organizing methods: classes and modules.

```ruby
class Order < ApplicationRecord

  has_many :line_items
  def self.find_all_unpaid
    self.where("paid = 0")
  end

  def total
    sum = 0
    line_items.each {|li| sum += li.total}
    sum
  end

end
```

- Class definitions start with the **class** keyword and are followed by the class name (which must start with an uppercase letter). This **Order** class is defined to be a subclass of the **ApplicationRecord** class. Prefixing a method name with **self** makes it a class method. Objects hold their state in *instance variables* whose names start with `@`. They aren't directly accessible from outside a class without methods that return their values.

```ruby
class Greeter
  def initialize(name)
    @name = name
  end

  def name
    @name
  end

  def name=(new_name)
    @name = new_name
  end
end

g = Greeter.new("Barney")
g.name    # => Barney
g.name = "Betty"
g.name    # => Betty
```

- Ruby also provides convenience methods that write accessor methods for you.

```ruby
class Greeter
  attr_accessor   :name       # create reader and writer methods
  attr_reader     :greeting   # create reader only
  attr_writer     :age        # create writer only
end
```

- Methods are public by default but can be declared `protected` or `private`.

```ruby
class MyClass
  def m1
  end
  protected
  def m2
  end
  private
  def m3
  end
end
```

- `private` methods can only be called within the same instance. `protected` methods can be called both in the same instance and by other instances of the same class and its subclasses.
- Modules also hold a collection of methods, constants, and other module and class definitions, but you can't create objects based on modules. Modules act as a namespace and allow you to share functionality among classes. If a class *mixes in* a module, that module's methods become available as if they'd been defined in the class. Multiple classes can mix in the same module to share the module's functionality without using inheritance. You can also mix multiple modules into a single class.
- Ruby uses modules for helper methods. Rails automatically mixes these helper modules into the appropriate view templates. For example, if you wanted to write a helper method that's callable from views invoked by the store controller, you could define the following module in the **store_helper.rb** file in the **app/helpers** directory:

```ruby
module StoreHelper
  def capitalize_word(string)
    string.split(" ").map {|word| word.capitalize}.join(" ")
  end
end
```

- In the context of Rails, YAML is used as a convenient way to define the configuration of things such as databases, test data, and translations; and the Ruby YAML module supports it.

```yaml
development:
  adapter: sqlite3
  database: storage/development.sqlite3
  pool: 5
  timeout: 5000
```

- In YAML, indentation is important, so this defines **development** as having a set of four key-value pairs separated by colons.

- *Marshaling* is the process of taking an object and converting it to a stream of bytes that can be stored outside the application. The saved object can be read by another instance of the application or a separate application to reconstitute the originally saved object. Some objects cannot be marshaled and raise a **TypeError**. When you load a marshaled object, Ruby needs to know the definition of the class of that objects and all the objects it contains. Rails uses marshaling to store session data. If you rely on Rails to dynamically load classes, it's possible that a particular class may not have been defined at the point it reconstitutes session data. For that reason, use the **model** declaration in your controllers to list all models that are marshaled to preemptively load the necessary classes.

```ruby
class CreateProducts < ActiveRecord::Migration[8.0]
  def change
    create_table :products do |t|
      t.string :title
      t.text :description
      t.decimal :price, precision: 8, scale: 2

      t.timestamps
    end
  end
end
```

- We are creating a `CreateProducts` class that inherits from the versioned `Migration` class in the `ActiveRecord` module specifying that compatibility with Rails 8 is desired. We define one method named `change`. This method calls the `create_table` method defined in `ActiveRecord::Migration`, passing it the name of the table in the form of a symbol. The call to `create_table` also passes as block to be evaluated before the table is created. The block is passed an object name `t` when called which is used to accumulate a list of fields. Methods added by Rails named after the common data types are used to add a field definition to the accumulating set of names. `decimal` also accepts a number of optional parameters, expressed as a hash.
- Ruby Idioms
  - Ruby method names can end in an exclamation mark (*a bang method*) or a question mark (*a predicate method*). Bang methods normally do something destructive to the receiver. Predicate methods normally return **true** or **false** depending on some condition.
  - `a || b` evaluates `a`. If it isn't **false** or **nil** then evaluation stops and `a` is returned. Otherwise, the statement returns `b`. This is a common way to return a default value if the first value isn't set.
  - `a ||= b` is the same as `a = a op b` for most operators.

```ruby
count += 1          # same as count = count + 1
price *= discount   #         price = price * discount
count ||= 0         #         count = count || 0
```

  - So `count ||= 0` gives `count` the value of 0 if `count` is nil or false.

- `obj = self.new` returns a new object of the receiver's class. This is more flexible than calling `Person.new` in case `Person` is subclassed.
- `lambda` converts a block into an object of type `Proc`. `->` is alternate syntax introduced in Ruby 1.9.

```
# These two are equivalent
square = lambda { |x| x * x }
square = ->(x) { x * x }
# Both can be called with `square.call(3)`, `square.(3)`, or `square[3]`.
```

- `require File.expand_path("../../config/environment",__FILE__)` loads an external source file into our application and is commonly used to include library code and classes.

Part 2 - Building an Application

Chapter 5 - The Depot Application

- Our shopping cart app has *buyers* and *sellers*. The *buyer* uses Depot to browse the products we have to sell, select some to purchase, and supply the information needed to create an order. A *seller* uses Depot to maintain a list of products to sell, to determine the orders that are awaiting shipment, and to mark orders as shipped.
- We can mock up some page flows by hand. Our buyer is going to get a catalog page that lists items. When they add an item to cart, they'll be taken to the cart, and can either return to shopping or checkout. When they do checkout, we capture contact and payment details then display a receipt page. The seller logs in and sees a menu letting her create or view a product or ship existing orders. When viewing a product, the seller can edit the details or delete the product entirely. Shipping is purposefully simplistic. It will display each order that hasn't shipped with one order per page, and the seller can either skip to the next or ship the order using the information from the page as appropriate.
- Based on the use cases and flows, we need some basic data. For one is the Product which at least has a name, description, image, and price. We need at least a login name and password for our seller. We'll have a cart since we need a place to keep a list of items the buyer bought, but it doesn't hold much other than that. Its line items will need at least product, quantity, and price. And for an Order we need buyer details, payment details, and shipping status.

Chapter 6 - Task A: Creating the Application

Iteration A1: Creating the Product Maintenance Application

- You can make a new project that uses Tailwind CSS with `rails new depot -css tailwind`.
- SQLite 3 is the default database for Rails development and requires no installation.
- In Rails, a model is automatically mapped to a database table whose name is the plural form of the model's class. So if we create a **Product** model, Rails associates it with a **products** table. Use the command to generate a *scaffold* for our new **Product** model.

```
depot> bin/rails generate scaffold Product \
         title:string description:text image:attachment price:decimal
```

- Note that `\` before a new line is used to continue lines in Linux/OS X, but you would need `^` in Windows (and backslash in `bin\rails`).
- The first file generated by scaffolding in `db/migrate` is the migration script `20250420000001_create_products.rb` (for some date).
- You can edit your migration scripts. We will modify our first one to specify `,precision: 8, scale: 2` for the decimal field.
- We also defined an attachment, so we need to install the tables that Active Storage uses to track attachments. This only needs to be done once per database and is done by `bin/rails active_storage:install`.
- To apply the migration, run `bin/rails db:migrate`. Rails looks for unapplied migrations and applies them. The **products** table is added to the database specified in the **development** section of the **config/database.yml** file.
- To undo and redo the last migration script run `bin/rails db:migrate:redo`
- When using things like CSS processors (Tailwind) and JavaScript bundlers, run your development server with `bin/dev`.
- Now we can go to http://localhost:3000/products and enter new products.
- But first let's modify `depot/app/views/products/_form.html.erb` to have more rows in the description and limit the acceptable files to select for upload to images:

```
# ...
<div class="my-5">
  <%= form.label :description %>
  <%= form.text_area :description, rows: 10, class:
    ["block shadow-sm rounded-md border px-3 py-2 mt-2 w-full",
    {"border-gray-400 focus:outline-blue-600":
      product.errors[:description].none?,
     "border-red-400 focus:outline-red-600":
      product.errors[:description].any?}] %>
</div>

<div class="my-5">
  <%= form.label :image %>
  <%= form.file_field :image, accept: "image/*", class:
    ["block shadow-sm rounded-md border px-3 py-2 mt-2 w-full",
    {"border-gray-400 focus:outline-blue-600": product.errors[:image].none?,
     "border-red-400 focus:outline-red-600": product.errors[:image].any?}] %>
</div>
# ...
```

- This looks like a big change, but we're only changing "rows: 4" to "rows: 10" and adding 'accept: "image/*"' to what's already there.

Iteration A2: Making Prettier Listings

- The product display is ugly. Make it look better and actually show the pictures instead of just the filenames.
- First we are going to modify the `db/seeds.rb` file to seed our database with more data instead of entering each product one at a time. Be warned that the `seeds.rb` script removes existing data before loading the new data. To populate your **products** table with test data run `bin/rails db:seed`.

```
# db/seeds.rb
Product.delete_all
# ...
product = Product.create(title: 'Rails Scales!',
  description:
    %(<p>
      <em>Practical Techniques for Performance and Growth</em>
      Rails doesn't scale. ...
    </p>),
  price: 30.95)

product.image.attach(io: File.open(
  Rails.root.join('db', 'images', 'cprpo.jpg')),
    filename: 'cprpo.jpg')

product.save!
# ...
```

- You can use `%(string)` as alternative syntax for double-quoted string literals.
- Now we can edit `app/views/products/index.html.erb` and replace the scaffold-generated view with Tailwind CSS styling:
- The Rails helper method `cycle` can alternate CSS classes of rows.
- The `truncate` method can be used to display the first eighty characters of the description, and the `strip_tags` method removes HTML tags.
- The `number_to_currency` helper is used to format money.
- The `image_tag` helper is used to display an image.
- You can put `data: { turbo_confirm: 'Are you sure?' }` to confirm a button press.
- The `mx-auto` class centers elements horizontally and the `px-2` and `px-3` classes add vertical and horizontal padding.
- Other classes have sensible names like `text-xl`, `bg-green-600`, and `hover:underline`.
- We also need to add a line to `app/controllers/application_controller.rb`:

```
class ApplicationController < ActionController::Base
  # Only allow modern browsers supporting webp images, web push, badges,
  # import maps, CSS nesting, and CSS :has.
  allow_browser versions: modern

  include ActiveStorage::SetCurrent
end
```

- Our newly formatted view has a problem that when you edit items the details don't show up until the user refreshes the page. We can do that with HotWire.

```
# app/models/product.rb
class Product < ApplicationRecord
  has_one_attached :image
  after_commit -> { broadcast_refresh_later_to "products" }
end

# app/views/products/index.html.erb - at the top
<%= turbo_stream_from "products" %>
<%= turbo_refreshes_with method: :morph, scroll: :preserve %>
```

- The first change tells Rails to broadcast changes to the product model to any clients that are listening. `after_commit` is an Active Record callback that's called after a record is saved. The first line of the second change tells the browser to listen for changes to the product model and update the page when they occur. The second line of the second change tells the browser to apply the changes directly to the page without refreshing it and to preserve the scroll position. Methods with names starting with `turbo_` are part of the Turbo framework. Turbo is a set of tools for building modern, reactive web applications.
- You can access the database directly with `bin/rails dbconsole`.
- To display application changes without reloading, add Hotwire Spark by running `bundle add hotwire-spark --group development`.

Chapter 7 - Task B: Validation and Unit Testing

Iteration B1: Validating!

- We want it so no product gets put in the database with an empty title or description field, and invalid URL for the image, or an invalid price. The model is an ideal place for validations

since all data goes through it.

- Add this line to `app/models/product.rb` in the `Product` class: `validates :title, :description, :image, presence: true`
- `validates` is the standard Rail validator that checks one or more model fields against one or more conditions. Here we are using `presence: true` to tell it to check that each of the named fields are present and non-empty.
- We also want to validate `:price`, so we add another validator: `validates :price, numericality: { greater_than_or_equal_to: 0.01 }`
- We test against 0.01 so smaller values aren't rounded down to zero.
- We can also add `validates :title, uniqueness: true` for unique titles and `validate :acceptable_image` if we define an `acceptable_image` method:

```ruby
# within the Product class of app/models/product.rb
def acceptable_image
  return unless image.attached?

  acceptable_types = [ "image/gif", "image/jpeg", "image/png" ]
  unless acceptable_types.include?(image.content_type)
    errors.add(:image, "must be a GIF, JPG or PNG image")
  end
end
```

Iteration B2: Unit Testing of Models

- There is already a `test/models/product_test.rb` from our model generation.
- The default `ProductTest` doesn't test anything.
- Tests subclass `ActiveSupport::TestCase` which itself is a subclass of `Minitest::Test` that comes preinstalled with Ruby. The `assert` method expects its arguments to be true. If we create an empty product, we expect it to be invalid and for an error to be associated with each field. We use the model's `errors` and `invalid?` methods to see if it validates, and we can use `any?` method of the error list to see if an error is associated with a particular attribute.

```ruby
test "product attributes must not be empty" do
  product = Product.new
  assert product.invalid?
  assert product.errors[:title].any?
  assert product.errors[:description].any?
  assert product.errors[:price].any?
  assert product.errors[:image].any?
end

test "product price must be positive" do
  product = Product.new(title:       "My Book Title",
                        description: "yyy")
  product.image.attach(io: File.open("test/fixtures/files/lorem.jpg"),
                       filename: "lorem.jpg", content_type: "image/jpeg")
  product.price = -1
  assert product.invalid?
  assert_equal [ "must be greater than or equal to 0.01" ],
    product.errors[:price]

  product.price = 0
  assert product.invalid?
  assert_equal [ "must be greater than or equal to 0.01" ],
    product.errors[:price]

  product.price = 1
  assert product.valid?
end
```

- Now to rerun just the unit tests run `rails test:models`.
- Assert statements take an optional extra parameter that is a string to be written along with the error message if the assertion fails.
- In Rails a test fixture is a specification of the initial contents of a model (or models) under test. So if we want **products** to always start with known data at the start of every unit test, we would specify the contents in a fixture. You specify fixture data in files in `test/fixtures` directory. The files contain test data in YAML format. Each fixture file contains the data for a single model. The base name of the file must match the name of a database table. So data for the **Product** model which is stored in the **products** table would be **products.yml**. Rails creates the fixture file when you create the model, so you just need to add data to it. The default fixtures are named `one` and `two` but it's highly recommended you use descriptive names for your fixtures. They aren't significant but are

referred to in tests using the fixtures. Note because we're using image attachments, we need to define blobs in `test/fixtures/active_storage/blobs.yml` and attach them to products in `fixtures/active_storage/attachments.yml`. For each fixture Rails loads into a test, it defines a method with the same name as the fixture you can use to access preloaded model objects containing the fixture data. Calling `products(:pragprog)` will return a `Product` model contain the data in the `pragprog` fixture.

Chapter 8 - Task C: Catalog Display

Iteration C1: Creating the Catalog Listing

- To make `store#index` the homepage of the site, we add a line of `root "store#index", as: "store_index"` to `config/routes.rb`. The `as: "store_index"` part tells it to create `store_index_path` and `store_index_url` accessor methods so existing code and test continue to work.
- If we know we will need the products from the model in our view, we can add `@products = Product.order(:title)` to get the products in our `app/controllers/store_controller.rb` `index` method. We will display them in order by title.

Iteration C2: Adding a Page Layout

- The file `views/layouts/application.html.erb` contains the standard page environment for the entire application. Editing it will change the look and feel of the entire site.

Iteration C3: Using a Helper to Format the Price

- Ruby provides an `sprintf` function that can format price, but instead we will use the `number_to_currency` helper function.

Iteration C4: Functional Testing of Controllers

- We can use `assert_select` to assert that certain css selectors are in an HTML response. Consider this expanded `test/controllers/store_controller_test.rb`:

```
require "test_helper"

class StoreControllerTest < ActionDispatch::IntegrationTest
  test "should get index" do
    get store_index_url
    assert_response :success
    assert_select "nav a", minimum: 4
    assert_select "main ul li", 3
    assert_select "h2", "The Pragmatic Programmer"
    assert_select "div", /\$[,\d]+\.\d\d/
  end
end
```

- Remember that selectors that start with a number sign (`#`) match on `id` attributes, selectors that start with a dot (`.`) match on class attributes, and selectors that contain no prefix match on element names.
- The first select test looks for an element name `a` that's contained in a `nav` element. The line after that verifies there are three `li` elements inside a `ul` which itself is the `main` element. The next line verifies there is an `h2` element with the title of the Ruby book from our fixture, and the fourth line verifies that the price is formatted correctly.
- The type of test that `assert_select` performs varies based on the type of the second parameter. When it is a number, it's treated as a quantity. A string is treated as an expected result. And a regular expression is matched against the value.
- Note both validation and functional tests will test the behavior of controllers but won't affect data already in the database retroactively. Our tests fixture has two rows with the same title, and that won't fail until we modify and save those records.

Iteration C5: Caching of Partial Results

- The `bin/rails dev:cache` command turns on caching in the development environment. We also add `<% cache products do %>` and `<% cache product do %>` lines to our store view to cache products.

Chapter 9 - Task D: Cart Creation

Iteration D1: Finding a Cart

- We will store the Cart in the database and associate it with a cart.id that we store in the session. To create the cart run `bin/rails generate scaffold Cart` and `bin/rails db:migrate`.

- Rails makes the current session look like a hash to the controller, so we can store the ID of the cart in the session by indexing it with the `:cart_id` symbol:

```
# depot/app/controllers/concerns/current_cart.rb
module CurrentCart
  private

    def set_cart
      @cart = Cart.find(session[:cart_id])
    rescue ActiveRecord::RecordNotFound
      @cart = Cart.create
      session[:cart_id] = @cart.id
    end
end
```

- The `app/controllers/concerns` directory is where we place common code to be shared among controllers. We also marked it `private` so Rails won't make it available as an action on the controllers.

Iteration D2: Connecting Products to Carts

- A cart contains line_items, so next we run `bin/rails generate scaffold LineItem product:references cart:belongs_to` and `bin/rails db:migrate`. Our new `LineItem` model is in `app/models/line_item.rb`. It has a `belongs_to` reference to both `:product` and `:cart` which produces an accessor method and tells Rails that rows in `line_items` are children of rows in `carts` and `products`. No line item can exist without corresponding `cart` and `product` rows. If a table has any columns whose values consist of ID values for another table (foreign keys), the corresponding model should have a `belongs_to` for each. Next we add a corresponding `has_many` to `app/models/cart.rb`:

```ruby
# app/models/cart.rb
class Cart < ApplicationRecord
  has_many :line_items, dependent: :destroy
end
```

- The `dependent: :destroy` line indicates if we destroy a cart, we want to destroy any line items that are associated with the cart. We also add a `has_many` to `app/models/product.rb` along with a `before_destroy :ensure_not_referenced_by_any_line_item` to make sure we don't delete products in carts after adding the private `ensure_not_referenced_by_any_line_item` method to `product.rb`. This is a hook method. A hook method is a method that Rails calls automatically at a given point in an object's life. This method is called `before_destroy` and if it throws `:abort`, the row isn't destroyed.

Iteration D3: Adding a Button

- Next we want to add an "Add to Cart" button under the item price in `app/views/store/index.html.erb`:

```erb
# app/views/store/index.html.erb

          <div>
            <%= number_to_currency(product.price) %>

            <%= button_to "Add to Cart",
              line_items_path(product_id: product),
              form_class: "inline",
              class: 'ml-4 rounded-lg py-1 px-2
                     text-white bg-green-600' %>
          </div>
```

- We also need some modifications in our line_items_controller:

```ruby
# app/controllers/line_items_controller.rb
class LineItemsController < ApplicationController
  include CurrentCart
  before_action :set_cart, only: %i[ create ]
  before_action :set_line_item, only: %i[ show edit update destroy ]

  # GET /line_items or /line_items.json
  # ...

  # POST /line_items or /line_items.json
  def create
    product = Product.find(params[:product_id])
    @line_item = @cart.line_items.build(product: product)

    respond_to do |format|
      if @line_item.save
        format.html { redirect_to @line_item.cart, notice: "Line item was
successfully created." }
        format.json { render :show, status: :created, location: @line_item }
      else
        format.html { render :new, status: :unprocessable_content }
        format.json { render json: @line_item.errors, status:
:unprocessable_content }
      end
    end
  end
```

- The `params` object holds all of the parameters passed in a browser request.
- We also didn't provide any attributes for our cart, so we need to update its erb to at least show the book title:

```erb
# app/views/carts/_cart.html.erb
<div id="<%= dom_id cart %>">
  <h2 class="font-bold text-lg mb-3">Your Pragmatic Cart</h2>

  <ul class="list-disc list-inside">
    <% cart.line_items.each do |item| %>
      <li><%= item.product.title %></li>
    <% end %>
  </ul>
</div>
```

- And we update our "should create line_item" test:

```
  test "should create line_item" do
    assert_difference("LineItem.count") do
      post line_items_url, params: { product_id: products(:pragprog).id }
    end

    follow_redirect!

    assert_select "h2", "Your Pragmatic Cart"
    assert_select "li", "The Pragmatic Programmer"
  end
```

Chapter 10 - Task E: A Smarter Cart

Iteration E1: Creating a Smarter Cart

- Adding a quantity to line_items is as simple as running: `bin/rails generate migration add_quantity_to_line_items quantity:integer`.
- Rails automatically looks for migrations with names of the form `AddXXXToTABLE` and `RemoveXXXFromTABLE`. The `XXX` part is ignored, so you still need to specify column names and when adding types.

Iteration E2: Handling Errors

- Rails defines a structure called a *flash*. The contents of the flash are available to the next request in this session before being deleted automatically. It's typically used to collect error messages. You can send messages in the flash with the `notice:` parameter of `redirect_to`.
- Every controller has a `logger` attribute, and you can log errors with `logger.error`. Logs are kept in the `log` directory.
- The `line_items_controller` has a private `line_item_params` method that defines what parameters are allowed.

Iteration E3: Finishing the Cart

Chapter 11 - Task F: Hotwiring the Storefront

- When you work with Hotwire, it's good to start with the non-Hotwire version of the application and then gradually introduce Hotwire features. The goal in this chapter is to

move the cart to a sidebar and only update that part of the page without triggering a whole redisplay.

Iteration F1: Moving the Cart

- Rails partial templates or *partials* are like a method for views. It's a chunk of a view in its own separate file. We currently iterate over each item in the cart and draw a row of the table. With partials we can pass a collection to the rendering method and it will automatically invoke the partial once for each item in the collection.
- A partial template is simply another template file (by default in the same directory as the object being rendered and with the name of the table as the name). Rails automatically prepends an _ to the name when looking for the file. So `<%= render cart.line_items %>` calls `_line_item.html.erb`. Within the template, we refer to the current object by the `line_item` variable which matches the name of the template.

Iteration F2: Creating a Hotwired Cart

- The trick when adding Turbo to an application is to take small steps. We will tell it to respond with a format to `turbo_stream` in `line_items_controller`:

```ruby
# app/controllers/line_items_controller.rb
  # POST /line_items or /line_items.json
  def create
    product = Product.find(params[:product_id])
    @line_item = @cart.add_product(product)

    respond_to do |format|
      if @line_item.save
        format.turbo_stream do
          render turbo_stream: turbo_stream.replace(
            :cart,
            partial: 'layouts/cart',
            locals: { cart: @cart }
          )
        end
        format.html { redirect_to store_index_url }
        format.json { render :show, status: :created, location: @line_item }
      else
        format.html { render :new, status: :unprocessable_content }
        format.json { render json: @line_item.errors, status:
:unprocessable_content }
      end
    end
  end
```

- This reads as "whenever we get a request that accepts a turbo stream response, we render a turbo stream response consisting of turbo stream replace specifying an HTML element ID of cart as the element to be replaced and rendering the partials which can be found in `app/views/layouts/_cart.html.erb` using the value of `@cart` as the value of `cart`." If the browser doesn't accept a turbo stream response (maybe JavaScript is disabled), it will still redirect to the store.

- So we have learned we need a partial for every area of the screen that we wish to dynamically update, the HTML in that partial needs to contain a unique HTML ID element, and we need to update the controller to return turbo streams. First we want to extract the "empty cart" notice to a partial.

- It is generally recommended to add a turbo_stream template like `app/views/line_items/create.turbo_stream.erb` when multiple items are in a response. Then we can change the block in our `line_items_controller.create` to the single line `format.turbo_stream`.

Iteration F3: Highlighting Changes

- We will use CSS animations to highlight the changes. A CSS animation loads when the page loads or when the class is applied to an element. This requires us to create a custom css asset in `app/assets/styleshseets/line_items.css`.

```
/* app/assets/stylesheets/line_items.css */
@keyframes line-item-highlight {
  0% {
    background: #8f8;
  }
  100% {
    background: none;
  }
}

.line-item-highlight {
  animation: line-item-highlight 1s;
}
```

- We also have to modify `app/controllers/line_items_controller.rb` so it knows the most recently added item. `format.turbo_stream` becomes `format.turbo_stream { @current_item = @line_item }` and then check `@current_item` in or `_line_item.html.erb` template to set the class.

Iteration F4: Broadcasting Updates with Action Cable

- Rails 5 introduced Action Cable which makes it easy to push data to browsers connected through WebSockets. We want our page to broadcast price updates. The customer has decided to honor the price at the time it was added to cart but wants the catalog display to be up-to-date.
- Using Action Cable is a three-step process: create a channel, broadcast some data, and receive the data. We can do most of the work with the rails generator: `bin/rails generate channel products`. We then add a `product.html.erb` template to `app/views/store` and use `turbo_frame_tag(dom_id(product))` helpers to create the HTML element and create a unique id for every product. Then we need to render the partial in our original `app/views/store/index.html.erb` and add a `turbo_stream_from 'products'` command to subscribe to the channel. But we also need to add a broadcast to `app/controllers/products_controller.rb`.

Chapter 12 - Task G: Check Out!

Iteration G1: Capturing an Order

- When scaffolding models, the default data type is `string` and doesn't need to be specified. `number` is a useful type to store discrete values, as you can define an `enum` in the model class. Regular columns default to nullable; add `null: false` to a migration if you want to require a value. The exception is `references` / `belongs_to` columns, which Rails makes non-null by default.
- Rails comes equipped with powerful form helper methods. A small example:

```
<%= form_with(model: order) do |form| %>
  <p>
    <%= form.label :name, "Name:" %>
    <%= form.text_field :name, size: 40 %>
  </p>
<% end %>
```

- Rails adds the `.field_with_errors` class to form field with errors.

Iteration G2: Adding Fields Dynamically to a Form

- To only display the proper fields for a selected payment type, we will need more JavaScript than that supported by Turbo. We will use Rails Stimulus.
- To run all tests including system tests run `bin/rails test:all`.

Iteration G3: Testing Our JavaScript Functionality

- `test/application_system_test_case.rb` controls which browser you test with and defaults to `:headless_chrome`.
- Be careful when writing tests with `has_no_field?` as there are an unlimited number of fields the app could lack so it is very sensitive to typos.
- You can run only the system tests with `bin/rails test:system`.

Chapter 13 - Task H: Sending Emails and Processing Payments Efficiently

- Sending email is a common task for any web application, and Rails has you covered with Action Mailer. It is built on Action Job which allows you to run code in a background process so the user doesn't have to wait for it to complete.

Iteration H1: Sending Confirmation Emails

- Sending email in Rails has three basic parts: configuring how email is to be sent, determining when to send the email, and specifying what you want to say.
- If you want the same email configuration for dev/test/prod, then put it in `config/environment.rb` in a `Rails.application.configure` block. Otherwise, put it in the appropriate environment file in `config/environments` directory.
- You need several statements in your `Rails.application.configure` block including `config.action_mailer.delivery_method = :smtp` (alternatives are `:sendmail` and `:test`). The `:test` setting is only for testing and appends the email to an array rather than sending it. It is the default for the `test` environment. The default in `development` is `:smtp` so be sure to set this to `:test` if you don't want to send out real emails when developing.
- Here are typical settings for Gmail:

```
Rails.application.configure do
  config.action_mailer.delivery_method = :smtp

  config.action_mailer.smtp_settings = {
    address:        "smtp.gmail.com",
    port:           587,
    domain:         "domain.of.sender.net",
    authentication: "plain",
    user_name:      "dave",
    password:       "secret",
    enable_starttls_auto: true
  }
end
```

- Like all configuration changes, this requires an application restart.
- A mailer is a class stored in the `app/mailers` directory with one or more methods that correspond to an email template. They use views to create the email message body. You can generate a mailer with: `bin/rails generate mailer Order received shipped`.
- `mail` takes `:to`, `:cc`, `:from`, and `:subject` parameters. You can set defaults at the top of your `app/mailers/order_mailer.rb` with lines such as `default from: "Sam Ruby <depot@example.com>".`

- You use `deliver_now` or `deliver_later` to send an email. Normally you don't want to make users wait for now. An example from the `orders_controller.rb` `create` method is `OrderMailer.received(@order).deliver_later` assuming you have wired the `order_mailer.rb` `received` method to take an `order` parameter.

Iteration H2: Connecting to a Slow Payment Processor with Active Job

- Here we're imagining we have a payment processor system, but it is slow. So we are moving the payment processing call to an Active Job task so we don't make the user wait.
- To make a `ChargeOrderJob`, we run `bin/rails generate job charge_order`.
- To queue a job using Active Job, use the `perform_later` method on the job class and pass it the arguments you want to be given to the `perform` method.

Chapter 14 - Task I: Logging In

Iteration I1: Authenticating Users

- Adding authentication is as simple as `bin/rails generate authentication`. This creates three models: `Session`, `User`, and `Current`. It creates controllers for sessions and passwords and a controller concern for authentication. Finally, it creates views for passwords and their associated mailer. It leaves to you the task of defining the user.
- We can generate a `User` scaffold with `bin/rails generate scaffold User name:string email_address:string password:digest --skip-collision-check --skip`. This adds `name` and respects the existing `email_address` and `password`. We also have to edit the migration script to add `name`.
- As part of generating authentication, a new gem is installed so a server restart is required.
- `rails console` invokes `irb` in the context of your application. We can use this to create a new user.
- There is another `config.action_mailer.delivery_method` option that is good for development. Set it to `:file` and it stores emails in `tmp/mails`.
- After adding authentication, all your tests will fail until you add a `login_as` helper method to `test/test_helper.rb` and call it in all your tests like `login_as users(:one)` in the `setup` method.

Iteration I2: Administration pages

Iteration I3: Permitting Access

- You can allow unauthenticated access to pages by putting the `allow_unauthenticated_access` method call in the controller.

Iteration I4: Adding a Sidebar, More Administration

- Active Record defines sixteen or so hook methods called at particular points in an object's life cycle.

Chapter 15 - Task J: Internationalization

Iteration J1: Selecting the Locale

- We start by creating a config file of what locales are available and which is the default:

```
# config/initializers/i18n.rb
#encoding: utf-8
I18n.default_locale = :en

LANGUAGES = [
  [ "English",                 "en"],
  [ "Espa&ntilde;ol".html_safe, "es"]
]
```

- This change requires a server restart.
- We are also going to modify our routes to include the locale. We are only going to internationalize the store pages and not admin pages. We put `:locale` in parentheses because it's optional.

```ruby
# config/routes.rb
Rails.application.routes.draw do
  get "admin" => "admin#index"
  get "up" => "rails/health#show", as: :rails_health_check

  resources :users
  resource :session
  resources :passwords, param: :token
  resources :products

  scope "(:locale)" do
    resources :orders
    resources :line_items
    resources :carts
    root "store#index", as: "store_index", via: :all
  end
end
```

- On a development server you can see all your routes at http://localhost:3000/rails/info/routes.
- We also want to add `before_action :set_i18n_locale_from_params` to our `app/controllers/application_controller.rb` base class that all controllers inherit from and define a `set_i18n_locale_from_params` function.

Iteration J2: Translating the Storefront

- Anything we want to translate, we pass to `i18n.translate` which is conveniently aliased as `I18n.t` and has a helper method of just `t`. You refer to your translation values like `<%= t('.title') %>` in the views.
- The mappings for `layouts/application.html.erb` are found in `config/locales/en.yml` for English under `en:`, `layouts:`, `application:`.
- You can use HTML codes like `ñ` or `á` if the value ends in `_html` in the `config/locales/es.yml` file.
- You can provide currency information in your translation file under `en:`, `number:`, `currency:`, `format`.

Iteration J3: Translating Checkout

- To translate error messages, you put translations under an `activerecord:` entry for `errors:`.

- We also used HTML codes in our errors, so you'll see in the project `_form.html.erb` partial where we displayed error strings `raw`.

Iteration J4: Adding a Locale Switcher

- We added a locale switcher dropdown form to the homepage.

Chapter 16 - Task K: Receive Emails and Respond with Rich Text

Iteration K1: Receiving Support Email with Action Mailbox

- Configuring Rails to receive emails requires three steps: initially setting up Action Mailbox, setting up Active Storage to hold the raw emails we receive, and implementing a *mailbox*, which is like a controller that handles incoming emails.
- Setting up Action Mailbox is just `bin/rails action_mailbox:install` then `bin/rails db:migrate`. In the real world, we'd also need to configure Action Mailbox for our particular incoming email service provider. See The Rails Guide.
- All incoming emails get stored in a cloud storage system like Amazon's S3.
- For development purposes, we're just going to use a disk-based storage service that works locally. Your dev and test environments are automatically configured for this `:local` storage with config in `config/storage.yml` and the server config files for the environments.
- We can configure a mailbox by putting `routing "support@example.com" => :support` in `app/mailboxes/application_mailbox.rb` to tell Rails any mail to `support@example.com` goes to `:support` then running `bin/rails generate mailbox support` to create the `SupportMailbox`. Rails calls `process` on `app/mailboxes/support_mailbox.rb` whenever a mail arrives and allows access to it through a `mail` variable.
- Rails includes Conductor for testing emails locally. You can see it at http://localhost:3000/rails/conductor/action_mailbox/inbound_emails.

Iteration K2: Storing Support Requests from Our Mailbox

- Rails deletes all emails after thirty days.
- We can use `receive_inbound_email_from_mail` in our tests.
- You can use erb syntax in fixtures. Also when it creates fixtures it uses the current time for `created_at` unless you specify otherwise.

Iteration K3: Responding with Rich Text

- You can use Action Text to edit and display Rich Text.

Chapter 17 - Task L: Deployment and Production

Iteration L1: Deploying Locally

- You start with a fine Dockerfile and can write a simple docker-compose.yml.

Iteration L2: Deployment to the Cloud

Iteration L3: Moving to Production

Chapter 18 - Depot Retrospective

- `rails stats` will show you a count of lines of code.

Part 3 - Rails in Depth

Note: These last few chapters are kinda dense with technical instructions and are worth having to review when working with Rails but not worth committing every detail to my personal notes. I went into some depth for Active Record, but took notes lightly for the rest.

Chapter 19 - Finding Your Way Around Rails

- To add `lib` to autoload paths and make it easy to `require` libraries put `config.autoload_paths += %W(#{Rails.root}/lib)` in your `config/application.rb`.
- You don't often need to use `require` in a Rails application if you follow standard naming conventions.
- The `admin/book` controller is in `app/controllers/admin/book_controller.rb` and named `Admin::BookController` with views in `app/views/admin/book`. It can be created with `bin/rails generate controller Admin::Book action1 action2 ...`.

Chapter 20 - Active Record

- Active Record is the object-relational mapping (ORM) provided by Rails.
- Any subclass of `ApplicationRecord` wraps a separate database table. By default, Active Record assumes that the name of the table associated with a given class is the plural form

of the name of that class. If the class name consists of multiple capitalized words, the table name is assumed to have underscores between these words. So `LineItem` is `line_items`. You can set a `self.table_name` attribute for the class if the defaults aren't fine for your purposes. You can also edit `config/initializers/inflections.rb` to add custom pluralization rules for your purposes.

- Instances of Active Record classes correspond to rows in a database table. They have attributes corresponding to the columns in the table. Rails determines these dynamically at run time. Active Record reflects on the schema inside the database to configure the classes that wrap tables.

- `created_at`, `created_on`, `updated_at`, and `updated_on` are special columns. Rails uses `_at` for columns containing times and `_on` for columns containing data. You add `created_at` and `updated_at` by specifying `t.timestamps` in your migration. Rails will keep these up to date.

- Rails also has `id` to contain the primary key and `xxx_id` to reference foreign keys to a table with the plural form of `xxx` as name. There is sometimes an `xxx_count` that is a counter cache for the child table `xxx`.

- You can specify another field as primary key with `self.primary_key = "xxxx"` in your class. If we override `id` as the primary key, we have to populate the new field ourselves rather than Rails providing a unique integer. We still set a value called `id` to do this. `id` is only used when setting the primary key and the column is still referred to by its name, whatever `xxxx` is.

- Rails defines `hash` as a reference to the primary key, so a model object can be a key in a hash provided it has been saved so the `id` is generated.

- Rails defines two model objects as equal (`==`) if they have the same class and primary key, regardless of data. This also means unsaved model objects always compare as equal no matter what their data since `id` isn't generated.

- Rails supports one-to-one, one-to-many, and many-to-many relationships. These are specified in the model class with `has_one`, `has_many`, `belongs_to`, and `has_and_belongs_to_many` declarations. The model for the table containing the foreign key always gets a `belongs_to` declaration. In a one-to-one, the other model will get a `has_one`. For example an `Order` could `has_one :invoice` and the `Invoice` would `belongs_to :order` and get the `order_id` field. An `Order` could `has_many :line_items` and the `LineItem` would `belongs_to :order` and get the `order_id` fields in each

`LineItem`. If you wanted a `Product` to have many `Category` and a `Category` to contain many `Product` you would put a `has_and_belongs_to_many :products` in `Category` and a `has_and_belongs_to_many :categories` in `Product` then Rails would create a `categories_products` join table associated `category_id` and `product_id` values. The name of the join table is the two table names in alphabetical order joined by `_`.

- CRUD: Create. You create rows by creating objects.

```ruby
an_order = Order.new
an_order.name     = "Dave Thomas"
an_order.email    = "dave@example.com"
an_order.address  = "123 Main St"
an_order.pay_type = "check"
an_order.save

# alternatively, you can create with block syntax without a new local
variable
Order.new do |o|
  o.name    = "Dave Thomas"
  # ...
  o.save
end

# You can also create with a hash of values
an_order = Order.new(
  name:     "Dave Thomas",
  email:    "dave@example.com",
  address:  "123 Main St",
  pay_type: "check")
an_order.save

# instantiate and save with `create`
an_order = Order.create(
  name:     "Dave Thomas",
  email:    "dave@example.com",
  address:  "123 Main St",
  pay_type: "check")

# create takes an array of hashes to create and array of model objects
orders = Order.create(
  [ { name:     "Dave Thomas",
      email:    "dave@example.com",
      address:  "123 Main St",
      pay_type: "check"
    },
    { name:     "Andy Hunt",
      email:    "andy@example.com",
      address:  "456 Gentle Drive",
      pay_type: "po"
    } ] )

# It is useful to use these hash methods with form parameters
@order = Order.new(order_params)
```

- The object only exists in memory until you call `save`. It also doesn't have an `id` until saved for the first time. The model object has a convenience method called `create` that both instantiates and saves the object and can be used instead of new.
- CRUD - Read. Every model class supports `find` which takes the primary key. `find` throws `ActiveRecord::RecordNotFound` if it can't find a record. `where` searches by other criteria and returns `nil` or an empty array if nothing is found.

```
an_order = Order.find(27)
product_list = Product.find(params[:product_ids])

# This returns a `ActiveRecord::Relation` of matching rows
pos = Order.where("name = 'Dave' and pay_type = 'po'")
name = params[:name]
pos = Order.where(["name = ? and pay_type ='po'", name])
pay_type = params[:pay_type]
pos = Order.where("name = :name and pay_type = :pay_type",
                  pay_type: pay_type, name: name)
# If you pass a hash as condition, Rails creates a where clause using the
# hash keys as column names and hash values as the values to match:
pos = Order.where(params[:order])  # Uses all order params
pos = Order.where(name: params[:name],
                  pay_type: params[:pay_type])
# Construct like values outside the query:
luser = User.where("name like ?", params[:name]+"%")
```

- The `where` clause returns an `ActiveRecord::Relation` of results. `first` returns the first object or `nil` if no matches. `all` returns all objects as an array of objects or an empty array. It also supports `each` and `map`. The query isn't actually called until one of these methods produces the results.
- The `order` method of the object lets you set return order. `limit` limits results. `offset` works with the `limit` method to paginate results. The `select` method allows you to `select` certain columns instead of `*`. `joins` allows you to join with other tables. `readonly` returns objects that can't be stored back into the database. (Older Rails versions automatically marked results of `joins` or `select` as `readonly`; that was removed in Rails 4, so in Rails 8 you have to call `.readonly` yourself if you want that behavior.) The `group` method adds a group by clause. `lock` allows you to specify to lock, optionally a specific type of lock passed as a string.

```
orders = Order.where(name: 'Dave').order("pay_type, shipped_at DESC")
orders = Order.where(name: 'Dave')
              .order("pay_type, shipped_at DESC")
              .limit(10)
prLineItems = LineItem.select('li.quantity').
  where("pr.title = 'The Pragmatic Programmer'").
  joins("as li inner join products as pr on li.product_id = pr.id")
summary = LineItem.select("sku, sum(amount) as amount").
                   group("sku")
```

- Rails can perform statistics on the values in a column through `average`, `maximum`, `minimum`, `sum`, and `count`. If combined with `group` they return a list of results. These also combine with `order` and `limit`.

```
average = Product.average(:price)
max     = Product.maximum(:price)
min     = Product.minimum(:price)
total   = Product.sum(:price)
number  = Product.count
# This returns an ordered hash
result = Order.group(:state).maximum(:amount)
puts result  #=> {"TX"=>12345, "NC"=>3456, ...}
# we have to use the SQLite aggregate function syntax below:
result = Order.group(:state).
               order("max(amount) desc").
               limit(3)
```

- You can add a `scope` associated with a `Proc` for reusable queries. Scopes aren't limited to `where` conditions. Chaining multiple `order` calls appends to the ORDER BY (use `reorder` to replace it outright instead), but chaining `limit` just overwrites the previous value, so don't stack those expecting them to combine.

```ruby
class Order < ActiveRecord::Base
  scope :last_n_days, ->(days) { where('update < ?', days) }
  # parameters are optional
  scope :checks, -> { where(pay_type: :check) }
end
orders = Order.last_n_days(7)
orders = Order.checks.last_n_days(7)
# You can combine a relation with a scope
in_house = Order.where('email LIKE "%@pragprog.com"')
in_house.checks.last_n_days(7)
```

- When the methods aren't flexible enough, `find_by_sql` takes a custom query. It returns an array of model objects that's potentially empty.
- Model objects have a `reload` method that refreshes values from the database.
- CRUD - Update. The `save` method stores changes to Active Record objects. Objects can only be saved if they include the `id` column. There is also an `update` method that lets you change attributes and save in one step. The model class also has an `update` method that takes an `id` and set of attributes and fetches, updates, and saves an object. This update can take an array of ids and array of attributes also. There is also an `update_all` method on the class that lets you use `set` and `where` clauses of the SQL update statement.

```ruby
order = Order.find(321)
order.update(name: "Barney", email: "barney@bedrock.com")
order = Order.update(321, name: "Barney", email: "barney@bedrock.com")
result = Product.update_all("price = 1.1*price", "title like %Java%")
```

- `save` returns `true` on success and `nil` otherwise. `save!` returns `true` on success and raises an exception otherwise. `create` returns the object and you need to check the object for validation errors to know if it succeeded. `create!` returns the object on success and raises an exception on failure.
- Class method `delete` takes an id or array of ids. `delete_all` deletes for a specified condition. `destroy` methods work on model objects. There is also a class level `destroy` which takes an id or array of ids and `destroy_all` which takes a condition. Unlike `delete` methods, `destroy` methods ensure Active Record callback and validation functions are invoked.

```
Order.delete(123)
User.delete([2,3,4,5])
Product.delete_all(["price > ?", @expensive_price])
```

- Active Record defines 16 callbacks; 14 are pairs of `before_` and `after_` such as `before_destroy` and `after_destroy`. `after_find` and `after_initialize` have no corresponding `before_` and are also different in other ways. You can specify a private or protected method as handler or pass a block after the declaration that receives the model object as parameter:

```
class Order < ActiveRecord::Base
  before_validation :normalize_credit_card_number
  after_create do |order|
    logger.info "Order #{order.id} created"
  end
  protected
  def normalize_credit_card_number
    self.cc_number.gsub!(/[-\s]/, '')
  end
end
```

- On save validations (covers both create and update):
    1. before_validation
    2. after_validation
    3. before_save
    4. around_save (code before yield runs here)
    5. before_create or before_update (whichever applies)
    6. around_create or around_update
    7. after_create or after_update
    8. after_save
    9. after_commit (or after_rollback if the transaction failed)
- On destroy validations:
    1. before_destroy
    2. around_destroy
    3. after_destroy

4. after_commit (or after_rollback)

- Any `before_*` callback can halt the operation by calling `throw :abort`
- `after_commit` / `after_rollback` sit outside the database transaction itself, everything above them runs inside one transaction that gets rolled back together if any step fails, which is why after_commit is the right place for side effects like your broadcast_refresh_later_to call or sending an email, work you don't want undone by a rollback but also don't want firing before the data is actually durable.
- You can also define a set of related callbacks in a handler class.
- You can execute a series of database actions in a `transaction` block called from a model class like `Account.transaction do`. You only need explicit transactions when you manage multiple SQL statements yourself.

Chapter 21 - Action Dispatch and Action Controller

- Action Pack consists of three modules. Action Dispatch routes requests to controllers. Action Controller converts requests to responses. Action View is used by Action Controller to format those responses.
- Declaring a route with `resources` causes seven new routes to be added.
- You can define `concerns` which are `resources` definitions that can be reused within other group of resources. For example, let's say you had `reviews`:

```
concern :reviewable do
  resources :reviews
end

resources :products, concern: :reviewable
resources :users, concern: :reviewable
```

- A controller always responds to the user exactly one time per request, so you should have just one call to a `render`, `redirect_to`, or `send_xxx` method. If no rendering is explicitly performed, the controller looks for a template named after the controller and action and automatically renders it.
- A Rails session is a hash-like structure that persists across requests and can hold any objects that can be marshaled. Sessions can be stored in cookies (the default) or the

database. Cookie-based sessions are limited to 4 KB. You should store large or volatile data in the database and then reference it from the session.

- The flash is a temporary scratchpad for values that is organized like a hash and stored in the session data. Values stored in the flash during a request will be available during the processing of the immediately following request.

- Callbacks enable you to write code in your controllers that wrap the processing performed by actions. Rails supports before, after, and around callbacks. A before callback halts the action by calling `throw(:abort)` (same mechanism as Active Record callbacks, above). Callbacks can also render output or redirect requests, in which case the associated action never happens. After callbacks can modify the outbound response, changing headers or content. An around callback invokes the action wherever the callback calls `yield` and if `yield` isn't called doesn't perform the action.

Chapter 22 - Action View

- Helpers for views associated with the `ProductController` tend to be placed in a helper module called `ProductHelper` in the file `product_helper.rb` found in the `app/helpers` directory. `rails generate controller` automatically generates a stub helper.

- The `debug` method dumps out its parameter to YAML and escapes the result so it can be displayed in an HTML page.

- If you're going to do much view work, review Rails standard helper methods and use them liberally.

- Use `link_to` to generate links, but use `button_to` for linking to actions that have side effects rather than specifying a `method:` for a `link_to`.

- It is easy to make sidebar menus where the current page name is show as plaintext and the other entries are hyperlinks:

```
<ul>
<% %w{ create list edit save logout }.each do |action| %>
  <li>
    <%= link_to_unless_current(action.capitalize, action: action) %>
  </li>
<% end %>
</ul>
```

- If a path to an image doesn't start with `/`, Rails assumes it is in the `app/assets/images` directory.

- If your current request is being handled by the `StoreController`, Rails will look for a layout called `store.html.erb` in the `app/views/layouts` directory. But if you have an `application.html.erb` layout, it will be used for all controllers without their own layout defined. You can turn off layouts by specifying `layout nil` in the controller or specify another layout with `layout "standard"` to specify `standard.html.erb`. If the parameter to `layout` is a symbol, Rails looks for a method on the controller of that name that returns the layout name to use. Subclasses of controllers use their parent's layout. You can also specify `render(layout: "standard")` or `render(layout: false)` to a `render` call.

- Partial template names begin with an underscore, but the underscore doesn't appear in the `render` call or when referring to variables passed in. Partials are assumed to live in the current controller's view directory unless the name is specified with one or more `/` characters. With slashes, it is looked for under `app/views`. Conventionally shared templates are stored in `app/views/shared` and rendered like `render(partial: "shared/post", object: @article)`. Remember, you refer to `@article` within the `_post.html.erb` template as `post` when it is passed through `object:`.

Chapter 23 - Migrations

- A migration is just a Ruby source file in your `db/migrate` directory that has a name beginning with a UTC timestamp to ensure consistent ordering. While they could be created by hand, it is easy to generate them through the Rails `bin/rails generate model model_name` or `bin/rails generate migration migration_name`. Migrations are run using `bin/rails db:migrate`. Rails keeps track of what is applied already in the `schema_migrations` table. Migrations may contain an `up` and `down` method, but it is generally smart enough to figure out how to undo operations if you only provide a `change` method.

- Migrations support the types `:binary`, `:boolean`, `:date`, `:datetime`, `:decimal`, `:float`, `:integer`, `:string`, `:text`, `:time`, and `:timestamp` and automatically map these to the correct type for the underlying database. `add_column` takes three options plus an additional two for decimal values. `null: true` or `false`, `limit: size`, and `default: value`. The default is calculated at time of creation, so don't expect `Time.now` to set the time of insert. Note that defaults are set on `save` to the database and will appear empty on new objects until reloaded.

```
add_column :orders, :attn, :string, limit: 100
add_column :orders, :order_type, :integer
add_column :orders, :ship_class, :string, null: false, default: 'priority'
add_column :orders, :amount, :decimal, precision: 8, scale: 2
```

- Migrations support `rename_column` and `change_column`. Rename is reversible. It may be better to `raise ActiveRecord::IrreversibleMigration` in the `down` script for `change_column` if you might lose data. There is also a reversible `rename_table` command. Indices can be added and removed via `add_index` and `remove_index`.

```
class CreateOrderHistories < ActiveRecord::Migration[8.0]
  def change
    create_table :order_histories do |t|
      t.integer :order_id, null: false
      t.text :notes

      t.timestamps
    end
  end
end
```

Chapter 24 - Customizing and Extending Rails

- Rails can be easily extended to allow you to define custom WebComponents to use in your pages with `lit`.
- To use RSpec instead of minitest, add `rspec-rails` to your Gemfile in the development and test groups. You can then use `bin/rails generate rspec:install` to install and `bin/rails spec` to run tests. `bin/rails generate rspec:model Cart` will create a spec for the `Cart` model. The default generators are also modified to create empty spec files in `spec/` instead of test files in `test/`. You can also run `bin/rails spec SPEC_OPTS="--format=doc"` to format spec output.
- It is easy to use Slim as an alternate templating language to reduce the verbosity of templates. Running `bundle add slim-rails` installs the gem and makes default generators produce slim files instead of ERB.